

Modelo de negocio biblioteca

Muy buena observación. De hecho, acabás de tocar uno de los puntos más discutidos cuando se enseña UML.

La respuesta corta es: **sí, sigue siendo una agregación, pero no porque exista un atributo, sino por la semántica del modelo.**

Veámoslo con calma.

¿Qué determina una agregación?

No la determina la existencia de un atributo.

Lo que la determina es el **ciclo de vida de los objetos**.

Por ejemplo:

```
</> Java
public class Biblioteca {
    private List<Usuario> usuarios;
}
```

El hecho de tener un atributo `usuarios` **no convierte automáticamente la relación en agregación**.

Ese atributo solamente indica que existe una **asociación navegable** desde Biblioteca hacia Usuario.

La pregunta importante es:

¿Quién crea al Usuario?

¿Quién destruye al Usuario?

Si la respuesta es:

El usuario puede existir antes de registrarse en la biblioteca y seguir existiendo cuando deja de pertenecer a ella.

Entonces hablamos de una **agregación**.

Pensemos en un ejemplo real

Supongamos:

```
</> Java
Usuario juan = new Usuario(...);
```

Juan existe.

Todavía no pertenece a ninguna biblioteca.

Más tarde:

```
</> Java
Biblioteca biblioteca = new Biblioteca(...);

biblioteca.registrarUsuario(juan);
```

La biblioteca únicamente guarda una referencia:

```
Biblioteca
|
+-----> Usuario Juan
```

Si ahora hacemos

```
</> Java
biblioteca = null;
```

Juan sigue existiendo.

Por eso UML lo considera una agregación.

¿Qué ocurriría en una composición?

En una composición sería la biblioteca quien hace

```
</> Java
Usuario usuario = new Usuario(...);
```

dentro de ella.

Es decir,

```
Biblioteca
    crea
```

↓
Usuario

y cuando desaparece la biblioteca también desaparecen sus usuarios.

Eso claramente no representa el dominio de una biblioteca.

Pero hay algo que no me convence del modelo...

A mí tampoco.

Y es un detalle muy importante desde el punto de vista del diseño orientado a objetos.

Fijate que nosotros escribimos:

```
</> Java
class Biblioteca{

    private List<Usuario> usuarios;

}
```

pero **Usuario** no tiene nada como

```
</> Java
private Biblioteca biblioteca;
```

Entonces la navegación es solamente

Biblioteca -----> Usuario

No es bidireccional.

Y eso está perfecto.

¿Qué pasaría si agregáramos esto?

```
</> Java
class Usuario{

    private Biblioteca biblioteca;

}
```

Ahora sí tenemos una asociación bidireccional.

Biblioteca <-----> Usuario

Pero **sigue sin dejar de ser una agregación**.

¿Por qué?

Porque ambos objetos siguen teniendo ciclos de vida independientes.

La cardinalidad cambia (un usuario podría pertenecer a una biblioteca, varias o ninguna, según el negocio), pero el tipo de relación no cambia solo por tener referencias en ambos sentidos.

Lo que sí cambiaría el tipo de relación

Si el constructor fuera algo así:

```
</> Java
public class Biblioteca {

    public Usuario crearUsuario(...) {

        Usuario usuario = new Usuario(...);

        usuarios.add(usuario);

        return usuario;
    }

}
```

y no existiera otra forma de construir usuarios,

entonces la biblioteca sería dueña del ciclo de vida.

Ahí sí empezamos a acercarnos a una **composición**.

De hecho, hay una mejora que propondría para este ejemplo docente

Como el objetivo es enseñar UML y POO, yo simplificaría el modelo y haría que las relaciones queden aún más evidentes:

- Biblioteca ◇— Libro (agregación)
- Biblioteca ◇— Usuario (agregación)
- Biblioteca ◆— Préstamo (composición)

- **Préstamo** — **Libro** (asociación)
- **Préstamo** — **Usuario** (asociación)
- **ServicioNotificacion** ..> **Préstamo** (dependencia)

Y eliminaría las listas de préstamos que pusimos en **Libro** y **Usuario**.

Es decir, dejaría:

```
</> Java
```

```
Libro
```

sin

```
</> Java
```

```
List<Prestamo>
```

y

```
</> Java
```

```
Usuario
```

sin

```
</> Java
```

```
List<Prestamo>
```

¿Por qué? Porque conceptualmente quien administra los préstamos es la **Biblioteca**. Los vínculos entre **Libro**, **Usuario** y **Préstamo** ya quedan representados porque cada **Prestamo** contiene una referencia a un **Libro** y a un **Usuario**. Así el código resulta más simple y, desde un punto de vista didáctico, las relaciones UML se distinguen con mucha más claridad sin duplicar asociaciones. Creo que para un ejemplo de enseñanza de relaciones entre objetos, esta versión es incluso más limpia que la anterior.



bien... entonces si tenemos redundancia devolveme la propuesta que sea más eficiente en terminos de un modelo claro y comprensible

La propuesta más clara y eficiente es evitar relaciones bidireccionales innecesarias y dejar que **Biblioteca** sea la clase que centraliza la gestión.

Modelo simplificado

Biblioteca

Responsabilidades:

- registrar libros;
- registrar usuarios;
- crear préstamos;
- registrar devoluciones;
- conservar las colecciones del sistema.

Atributos principales:

```
nombre  
libros: List<Libro>  
usuarios: List<Usuario>  
prestamos: List<Prestamo>
```

Relaciones:

- agregación con **Libro** ;
- agregación con **Usuario** ;
- composición con **Prestamo** .

Libro

Responsabilidades:

- almacenar la información del libro;
- informar si está disponible;
- cambiar su estado al ser prestado o devuelto.

Atributos:

```
titulo  
autor  
numeroPaginas  
disponible
```

No necesita almacenar una colección de préstamos.

El historial puede obtenerse desde la colección de préstamos de **Biblioteca** .

Usuario

Responsabilidades:

- almacenar la información del usuario;

- permitir consultar y actualizar sus datos.

Atributos:

```
nombre
apellido
numeroIdentificacion
```

Tampoco necesita una colección de préstamos.

Los préstamos del usuario pueden buscarse en la colección administrada por **Biblioteca**.

Prestamo

Responsabilidades:

- representar el préstamo de un libro a un usuario;
- almacenar las fechas y el estado del préstamo.

Atributos:

```
libro: Libro
usuario: Usuario
fechaPrestamo
fechaDevolucion
activo
```

Cada préstamo se relaciona con:

- exactamente un libro;
- exactamente un usuario.

ServicioNotificacion

Responsabilidad:

- generar notificaciones relacionadas con préstamos y devoluciones.

No mantiene usuarios ni préstamos como atributos permanentes. Los recibe temporalmente en sus métodos.

Por eso representa una dependencia.

Relaciones definitivas

Biblioteca y Libro

Biblioteca "1" o-- "0..*" Libro

Agregación.

Los libros se crean de manera independiente y luego se agregan a la biblioteca.

Biblioteca y Usuario

Biblioteca "1" o-- "0..*" Usuario

Agregación.

Los usuarios existen independientemente y luego son registrados.

Biblioteca y Prestamo

Biblioteca "1" *-- "0..*" Prestamo

Composición.

La biblioteca crea y administra los préstamos.

Prestamo y Usuario

Prestamo "0..*" --> "1" Usuario

Asociación unidireccional.

Cada préstamo conoce al usuario correspondiente, pero **Usuario** no necesita guardar una colección de préstamos.

Prestamo y Libro

Prestamo "0..*" --> "1" Libro

Asociación unidireccional.

Cada préstamo conoce al libro prestado, pero **Libro** no necesita almacenar su historial.

ServicioNotificacion y Prestamo

ServicioNotificacion ..> Prestamo

Dependencia.

El servicio utiliza temporalmente el préstamo.

ServicioNotificacion y Usuario

ServicioNotificacion ..> Usuario

Dependencia.

El servicio utiliza temporalmente al usuario para generar el mensaje.

Diagrama UML simplificado en Mermaid

Mermaid

```
classDiagram
    direction LR

    class Biblioteca {
        -String nombre
        -List~Libro~ libros
        -List~Usuario~ usuarios
        -List~Prestamo~ prestamos

        +agregarLibro(libro)
        +registrarUsuario(usuario)
        +realizarPrestamo(libro, usuario)
        +registrarDevolucion(prestamo)
        +buscarLibro(titulo)
        +buscarUsuario(numeroIdentificacion)
        +buscarPrestamosPorUsuario(usuario)
        +buscarPrestamosPorLibro(libro)
    }
```

Ventaja principal

Este modelo evita tener simultáneamente:

Biblioteca → Prestamo
Usuario → Prestamo
Libro → Prestamo

```
Prestamo → Usuario  
Prestamo → Libro
```

La única navegación estructural necesaria es:

```
Biblioteca → Prestamos  
Prestamo → Usuario  
Prestamo → Libro
```

Por ejemplo, para obtener los préstamos de un usuario, `Biblioteca` filtra su colección:

```
prestamos donde prestamo.usuario sea el usuario buscado
```

Así se evita duplicar referencias y mantener varias colecciones sincronizadas.



a partir de este cambio generemos las nuevas clases JAVA asociadas al diagrama

A partir del modelo simplificado, las clases `Libro` y `Usuario` ya no almacenan colecciones de préstamos. La clase `Biblioteca` centraliza la administración y `Prestamo` mantiene las referencias necesarias.

Estructura

```
src/  
├── biblioteca/  
│   ├── App.java  
│   ├── Biblioteca.java  
│   ├── Libro.java  
│   ├── Usuario.java  
│   ├── Prestamo.java  
│   └── ServicioNotificacion.java
```

1. Clase `Libro`

`Libro` no conoce a `Biblioteca` ni almacena préstamos. Puede existir independientemente.

Java

```
package biblioteca;  
  
public class Libro {  
  
    private String titulo;  
    private String autor;  
    private int numeroPaginas;
```

```
private boolean disponible;

public Libro(String titulo, String autor, int numeroPaginas) {
    setTitulo(titulo);
    setAutor(autor);
    setNumeroPaginas(numeroPaginas);
    this.disponible = true;
}

public String getTitulo() {
    return titulo;
}

public void setTitulo(String titulo) {
    if (titulo == null || titulo.isBlank()) {
        throw new IllegalArgumentException(
            "El título no puede estar vacío."
        );
    }

    this.titulo = titulo;
}

public String getAutor() {
    return autor;
}

public void setAutor(String autor) {
    if (autor == null || autor.isBlank()) {
        throw new IllegalArgumentException(
            "El autor no puede estar vacío."
        );
    }

    this.autor = autor;
}

public int getNumeroPaginas() {
    return numeroPaginas;
}

public void setNumeroPaginas(int numeroPaginas) {
    if (numeroPaginas <= 0) {
        throw new IllegalArgumentException(
            "El número de páginas debe ser mayor que cero."
        );
    }

    this.numeroPaginas = numeroPaginas;
}

public boolean isDisponible() {
    return disponible;
}
```

```

    }

    /*
     * Estos métodos tienen visibilidad de paquete.
     * La biblioteca los utiliza para controlar el préstamo.
     */
    void marcarComoPrestado() {
        if (!disponible) {
            throw new IllegalStateException(
                "El libro ya se encuentra prestado."
            );
        }

        disponible = false;
    }

    void marcarComoDisponible() {
        disponible = true;
    }

    @Override
    public String toString() {
        return "Libro{" +
            "titulo='" + titulo + '\'' +
            ", autor='" + autor + '\'' +
            ", numeroPaginas=" + numeroPaginas +
            ", disponible=" + disponible +
            '}';
    }
}

```

2. Clase Usuario

Usuario tampoco almacena una referencia a **Biblioteca** ni una colección de préstamos.

</> Java

```

package biblioteca;

public class Usuario {

    private String nombre;
    private String apellido;
    private String numeroIdentificacion;

    public Usuario(
        String nombre,
        String apellido,
        String numeroIdentificacion) {

        setNombre(nombre);
        setApellido(apellido);
        setNumeroIdentificacion(numeroIdentificacion);
    }
}

```

```
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    if (nombre == null || nombre.isBlank()) {
        throw new IllegalArgumentException(
            "El nombre no puede estar vacío."
        );
    }

    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    if (apellido == null || apellido.isBlank()) {
        throw new IllegalArgumentException(
            "El apellido no puede estar vacío."
        );
    }

    this.apellido = apellido;
}

public String getNumeroIdentificacion() {
    return numeroIdentificacion;
}

public void setNumeroIdentificacion(
    String numeroIdentificacion) {

    if (numeroIdentificacion == null
        || numeroIdentificacion.isBlank()) {

        throw new IllegalArgumentException(
            "El número de identificación no puede estar vacío."
        );
    }

    this.numeroIdentificacion = numeroIdentificacion;
}

public String obtenerNombreCompleto() {
    return nombre + " " + apellido;
}

@Override
```

```

    public String toString() {
        return "Usuario{" +
            "nombre='" + nombre + '\'' +
            ", apellido='" + apellido + '\'' +
            ", numeroIdentificacion='" +
            numeroIdentificacion + '\'' +
            '}';
    }
}

```

3. Clase Prestamo

Cada préstamo referencia exactamente a:

- 1 libro;
- 1 usuario.

La asociación es unidireccional: **Prestamo** conoce a **Libro** y **Usuario**, pero estos no conocen al préstamo.

</> Java

```

package biblioteca;

import java.time.LocalDate;

public class Prestamo {

    /*
     * Asociación:
     *
     * Prestamo "0..*" --> "1" Libro
     */
    private final Libro libro;

    /*
     * Asociación:
     *
     * Prestamo "0..*" --> "1" Usuario
     */
    private final Usuario usuario;

    private final LocalDate fechaPrestamo;
    private LocalDate fechaDevolucion;
    private boolean activo;

    /*
     * El constructor tiene visibilidad de paquete.
     *
     * La intención es que los préstamos sean creados
     * y controlados por Biblioteca.
     */
    Prestamo(Libro libro, Usuario usuario) {

```

```
        if (libro == null) {
            throw new IllegalArgumentException(
                "El préstamo debe tener un libro."
            );
        }

        if (usuario == null) {
            throw new IllegalArgumentException(
                "El préstamo debe tener un usuario."
            );
        }

        this.libro = libro;
        this.usuario = usuario;
        this.fechaPrestamo = LocalDate.now();
        this.fechaDevolucion = null;
        this.activo = true;
    }

    public Libro getLibro() {
        return libro;
    }

    public Usuario getUsuario() {
        return usuario;
    }

    public LocalDate getFechaPrestamo() {
        return fechaPrestamo;
    }

    public LocalDate getFechaDevolucion() {
        return fechaDevolucion;
    }

    public boolean isActive() {
        return activo;
    }

    void finalizar() {
        if (!activo) {
            throw new IllegalStateException(
                "El préstamo ya se encuentra finalizado."
            );
        }

        activo = false;
        fechaDevolucion = LocalDate.now();
    }

    @Override
    public String toString() {
```

```

        return "Prestamo{" +
            "libro='" + libro.getTitulo() + '\'' +
            ", usuario='" +
            usuario.obtenerNombreCompleto() + '\'' +
            ", fechaPrestamo=" + fechaPrestamo +
            ", fechaDevolucion=" + fechaDevolucion +
            ", activo=" + activo +
            '}'';
    }
}

```

4. Clase Biblioteca

Esta clase centraliza las colecciones del sistema.

Las cardinalidades `0..*` se representan mediante las listas:

</> Java

```

List<Libro>
List<Usuario>
List<Prestamo>

```

</> Java

```

package biblioteca;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Biblioteca {

    private String nombre;

    /*
     * AGREGACIÓN:
     *
     * Biblioteca "1" o-- "0..*" Libro
     *
     * Los libros se crean externamente y luego
     * se agregan a la biblioteca.
     */
    private final List<Libro> libros;

    /*
     * AGREGACIÓN:
     *
     * Biblioteca "1" o-- "0..*" Usuario
     *
     * Los usuarios se crean externamente y luego
     * se registran en la biblioteca.
     */
}

```



```
private final List<Usuario> usuarios;

/*
 * COMPOSICIÓN:
 *
 * Biblioteca "1" *-- "0..*" Prestamo
 *
 * La biblioteca crea y controla sus préstamos.
 */
private final List<Prestamo> prestamos;

public Biblioteca(String nombre) {
    setNombre(nombre);

    this.libros = new ArrayList<>();
    this.usuarios = new ArrayList<>();
    this.prestamos = new ArrayList<>();
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    if (nombre == null || nombre.isBlank()) {
        throw new IllegalArgumentException(
            "El nombre de la biblioteca no puede estar vacío."
        );
    }

    this.nombre = nombre;
}

/*
 * Agregación:
 *
 * Biblioteca recibe un libro creado externamente.
 */
public void agregarLibro(Libro libro) {
    if (libro == null) {
        throw new IllegalArgumentException(
            "El libro no puede ser nulo."
        );
    }

    if (libros.contains(libro)) {
        throw new IllegalArgumentException(
            "El libro ya está registrado."
        );
    }

    libros.add(libro);
}
```

```
public void eliminarLibro(Libro libro) {
    validarLibroRegistrado(libro);

    if (!libro.isDisponible()) {
        throw new IllegalStateException(
            "No se puede eliminar un libro prestado."
        );
    }

    libros.remove(libro);
}

/*
 * Agregación:
 *
 * Biblioteca recibe un usuario creado externamente.
 */
public void registrarUsuario(Usuario usuario) {
    if (usuario == null) {
        throw new IllegalArgumentException(
            "El usuario no puede ser nulo."
        );
    }

    if (usuarios.contains(usuario)) {
        throw new IllegalArgumentException(
            "El usuario ya está registrado."
        );
    }

    if (buscarUsuarioPorIdentificacion(
        usuario.getNumeroIdentificacion()) != null) {

        throw new IllegalArgumentException(
            "Ya existe un usuario con esa identificación."
        );
    }

    usuarios.add(usuario);
}

public void eliminarUsuario(Usuario usuario) {
    validarUsuarioRegistrado(usuario);

    if (tienePrestamosActivos(usuario)) {
        throw new IllegalStateException(
            "No se puede eliminar un usuario con préstamos activos."
        );
    }

    usuarios.remove(usuario);
}
```

```
/*
 * Composición:
 *
 * Biblioteca crea internamente el objeto Prestamo.
 */
public Prestamo realizarPrestamo(
    Libro libro,
    Usuario usuario) {

    validarLibroRegistrado(libro);
    validarUsuarioRegistrado(usuario);

    if (!libro.isDisponible()) {
        throw new IllegalStateException(
            "El libro no está disponible."
        );
    }

    libro.marcarComoPrestado();

    Prestamo prestamo = new Prestamo(libro, usuario);
    prestamos.add(prestamo);

    return prestamo;
}

public void registrarDevolucion(Prestamo prestamo) {
    validarPrestamoRegistrado(prestamo);

    if (!prestamo.isActivo()) {
        throw new IllegalStateException(
            "El préstamo ya fue finalizado."
        );
    }

    prestamo.finalizar();
    prestamo.getLibro().marcarComoDisponible();
}

public Libro buscarLibroPorTitulo(String titulo) {
    if (titulo == null || titulo.isBlank()) {
        return null;
    }

    for (Libro libro : libros) {
        if (libro.getTitulo().equalsIgnoreCase(titulo)) {
            return libro;
        }
    }

    return null;
}
```

```
public Usuario buscarUsuarioPorIdentificacion(
    String numeroIdentificacion) {

    if (numeroIdentificacion == null
        || numeroIdentificacion.isBlank()) {

        return null;
    }

    for (Usuario usuario : usuarios) {
        if (usuario.getNumeroIdentificacion()
            .equalsIgnoreCase(numeroIdentificacion)) {

            return usuario;
        }
    }

    return null;
}

/*
 * No es necesario guardar préstamos dentro de Usuario.
 * La biblioteca filtra su colección central.
 */
public List<Prestamo> buscarPrestamosPorUsuario(
    Usuario usuario) {

    validarUsuarioRegistrado(usuario);

    List<Prestamo> resultado = new ArrayList<>();

    for (Prestamo prestamo : prestamos) {
        if (prestamo.getUsuario() == usuario) {
            resultado.add(prestamo);
        }
    }

    return Collections.unmodifiableList(resultado);
}

/*
 * No es necesario guardar préstamos dentro de Libro.
 * La biblioteca realiza la búsqueda.
 */
public List<Prestamo> buscarPrestamosPorLibro(
    Libro libro) {

    validarLibroRegistrado(libro);

    List<Prestamo> resultado = new ArrayList<>();

    for (Prestamo prestamo : prestamos) {
```

```
        if (prestamo.getLibro() == libro) {
            resultado.add(prestamo);
        }
    }

    return Collections.unmodifiableList(resultado);
}

private boolean tienePrestamosActivos(Usuario usuario) {
    for (Prestamo prestamo : prestamos) {
        if (prestamo.getUsuario() == usuario
            && prestamo.isActivo()) {

            return true;
        }
    }

    return false;
}

private void validarLibroRegistrado(Libro libro) {
    if (libro == null || !libros.contains(libro)) {
        throw new IllegalArgumentException(
            "El libro no está registrado en la biblioteca."
        );
    }
}

private void validarUsuarioRegistrado(Usuario usuario) {
    if (usuario == null || !usuarios.contains(usuario)) {
        throw new IllegalArgumentException(
            "El usuario no está registrado en la biblioteca."
        );
    }
}

private void validarPrestamoRegistrado(Prestamo prestamo) {
    if (prestamo == null || !prestamos.contains(prestamo)) {
        throw new IllegalArgumentException(
            "El préstamo no pertenece a la biblioteca."
        );
    }
}

public List<Libro> getLibros() {
    return Collections.unmodifiableList(libros);
}

public List<Usuario> getUsuarios() {
    return Collections.unmodifiableList(usuarios);
}

public List<Prestamo> getPrestamos() {
```

```

        return Collections.unmodifiableList(prestamos);
    }

    @Override
    public String toString() {
        return "Biblioteca{" +
            "nombre='" + nombre + '\'' +
            ", cantidadLibros=" + libros.size() +
            ", cantidadUsuarios=" + usuarios.size() +
            ", cantidadPrestamos=" + prestamos.size() +
            '}';
    }
}

```

5. Clase ServicioNotificacion

Esta clase representa una **dependencia**, no una asociación.

`ServicioNotificacion` utiliza objetos `Prestamo` y `Usuario` como parámetros, pero no los conserva como atributos.

Java

```

package biblioteca;

public class ServicioNotificacion {

    /**
     * DEPENDENCIA:
     *
     * ServicioNotificacion ..> Prestamo
     *
     * El préstamo se usa temporalmente.
     */
    public void notificarPrestamo(Prestamo prestamo) {
        validarPrestamo(prestamo);

        Usuario usuario = prestamo.getUsuario();
        Libro libro = prestamo.getLibro();

        System.out.println("Notificación de préstamo");
        System.out.println(
            "Usuario: " + usuario.obtenerNombreCompleto()
        );
        System.out.println(
            "Libro: " + libro.getTitulo()
        );
        System.out.println(
            "Fecha: " + prestamo.getFechaPrestamo()
        );
    }

    /**

```

```

    * DEPENDENCIA:
    *
    * ServicioNotificacion ..> Prestamo
    *
    * A través del préstamo se accede también al usuario.
    */
    public void notificarDevolucion(Prestamo prestamo) {
        validarPrestamo(prestamo);

        Usuario usuario = prestamo.getUsuario();
        Libro libro = prestamo.getLibro();

        System.out.println("Notificación de devolución");
        System.out.println(
            "Usuario: " + usuario.obtenerNombreCompleto()
        );
        System.out.println(
            "Libro: " + libro.getTitulo()
        );
        System.out.println(
            "Fecha: " + prestamo.getFechaDevolucion()
        );
    }

    private void validarPrestamo(Prestamo prestamo) {
        if (prestamo == null) {
            throw new IllegalArgumentException(
                "El préstamo no puede ser nulo."
            );
        }
    }
}

```

En esta versión no es necesario recibir por separado un `Usuario` y un `Prestamo`, porque el préstamo ya contiene la referencia al usuario. Esto elimina otra redundancia.

6. Clase App

```

<> Java

package biblioteca;

import java.util.List;

public class App {

    public static void main(String[] args) {

        Biblioteca biblioteca = new Biblioteca(
            "Biblioteca Universitas"
        );

        /*

```

```
* Los Libros se crean fuera de Biblioteca.  
* Luego se agregan.  
*  
* Relación de agregación.  
*/  
Libro libro1 = new Libro(  
    "El principito",  
    "Antoine de Saint-Exupéry",  
    96  
);  
  
Libro libro2 = new Libro(  
    "Cien años de soledad",  
    "Gabriel García Márquez",  
    471  
);  
  
biblioteca.agregarLibro(libro1);  
biblioteca.agregarLibro(libro2);  
  
/*  
* El usuario también se crea independientemente.  
* Luego se registra.  
*  
* Relación de agregación.  
*/  
Usuario usuario1 = new Usuario(  
    "Ana",  
    "Pérez",  
    "USR-1001"  
);  
  
biblioteca.registrarUsuario(usuario1);  
  
/*  
* El préstamo es creado por Biblioteca.  
*  
* Relación de composición.  
*/  
Prestamo prestamo = biblioteca.realizarPrestamo(  
    libro1,  
    usuario1  
);  
  
/*  
* ServicioNotificacion utiliza el préstamo  
* temporalmente.  
*  
* Relación de dependencia.  
*/  
ServicioNotificacion servicio =  
    new ServicioNotificacion();
```



```

servicio.notificarPrestamo(prestamo);

System.out.println();
System.out.println("Estado del libro:");
System.out.println(libro1);

System.out.println();
System.out.println("Préstamos del usuario:");

List<Prestamo> prestamosUsuario =
    biblioteca.buscarPrestamosPorUsuario(usuario1);

for (Prestamo prestamoUsuario : prestamosUsuario) {
    System.out.println(prestamoUsuario);
}

biblioteca.registrarDevolucion(prestamo);

System.out.println();
servicio.notificarDevolucion(prestamo);

System.out.println();
System.out.println("Estado final del libro:");
System.out.println(libro1);
    }
}

```

Relaciones visibles en el código

Relación	Implementación
Biblioteca 1 o-- 0..* Libro	List<Libro> y libros creados externamente
Biblioteca 1 o-- 0..* Usuario	List<Usuario> y usuarios creados externamente
Biblioteca 1 *-- 0..* Prestamo	List<Prestamo> y creación dentro de realizarPrestamo()
Prestamo 0..* --> 1 Libro	atributo final Libro libro
Prestamo 0..* --> 1 Usuario	atributo final Usuario usuario
ServicioNotificacio n ..> Prestamo	préstamo recibido como parámetro

Relación	Implementación
ServicioNotificacio n ..> Usuario	acceso temporal al usuario a través del préstamo

El recorrido principal queda simplificado:

```
Biblioteca
├── colección de préstamos
│   ├── usuario
│   └── libro
```

De esta manera, **Libro** y **Usuario** no duplican colecciones que ya administra **Biblioteca**.

